

AI ASSISTANT CODING (ASSIGNMENT-13.5)

NAME : M.Sai Charani

HALL TICKET NO : 2303A52192

BATCH : 34

Task Description #1 (Refactoring – Removing Global Variables)

- Task: Use AI to eliminate unnecessary global variables from the code.

- **Instructions:**

- o Identify global variables used across functions.
- o Refactor the code to pass values using function parameters.
- o Improve modularity and testability.

- Sample Legacy Code:

```
rate = 0.1  
  
def calculate_interest(amount):  
    return amount * rate  
  
print(calculate_interest(1000))
```

- **Expected Output:**

- o Refactored version passing rate as a parameter or using a configuration structure.

Prompt : Act as a senior Python software engineer.

Refactor the following legacy Python code to remove global variables and improve modularity.

Requirements:

1. Eliminate the global variable.
2. Pass dependencies as function parameters.
3. Add type hints and docstrings.
4. Follow PEP 8 conventions.

5. Ensure output remains identical.

6. Briefly explain why removing global state improves testability.

Legacy Code:

rate = 0.1

def calculate_interest(amount):

return amount * rate

print(calculate_interest(1000))

Provide:

- Refactored code
- Explanation of improvements



Task Description #2 : (Refactoring Deeply Nested Conditionals)

- Task: Use AI to refactor deeply nested if–elif–else logic into a cleaner structure.

- Focus Areas:

- o Readability

- o Logical simplification

- o Maintainability

Legacy Code:

score = 78

if score >= 90:

```
print("Excellent")  
  
else:  
  
if score >= 75:  
    print("Very Good")  
  
else:  
  
if score >= 60:  
    print("Good")  
  
else:  
  
print("Needs Improvement")
```

Expected Outcome:

o Flattened logic using guard clauses or a mapping-based approach.

Prompt :

Act as a clean-code expert in Python.

Refactor the following deeply nested conditional logic.

Requirements:

1. Remove nested if-else blocks.
2. Use guard clauses OR a clean mapping-based solution.
3. Improve readability and maintainability.
4. Keep the output behavior exactly the same.
5. Add comments explaining the logic.
6. Briefly explain why the new version is cleaner.

Legacy Code:

```
score = 78  
  
if score >= 90:  
    print("Excellent")
```

else:

if score >= 75:

print("Very Good")

else:

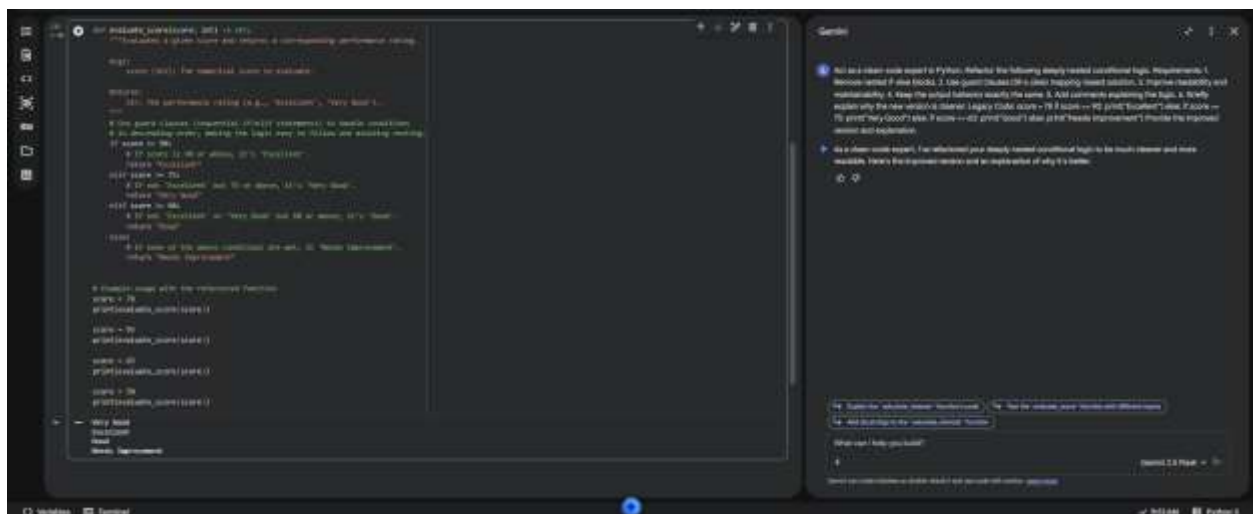
if score >= 60:

print("Good")

else:

print("Needs Improvement")

Provide the improved version and explanation.



Task 3 (Refactoring Repeated File Handling Code)

- Task: Use AI to refactor repeated file open/read/close logic.
- Focus Areas:
 - o DRY principle
 - o Context managers
 - o Function reuse

Legacy Code:

```
f = open("data1.txt")
print(f.read())
f.close()

f = open("data2.txt")
print(f.read())
f.close()
```

Expected Outcome:

o Reusable function using with open() and parameters.

Prompt:

Act as a Python best-practices specialist.

Refactor the following code to eliminate repetition and improve file handling safety.

Requirements:

1. Apply the DRY principle.
2. Use context managers (with open()).
3. Create a reusable function.
4. Add proper error handling.
5. Use type hints and docstrings.
6. Keep functionality unchanged.

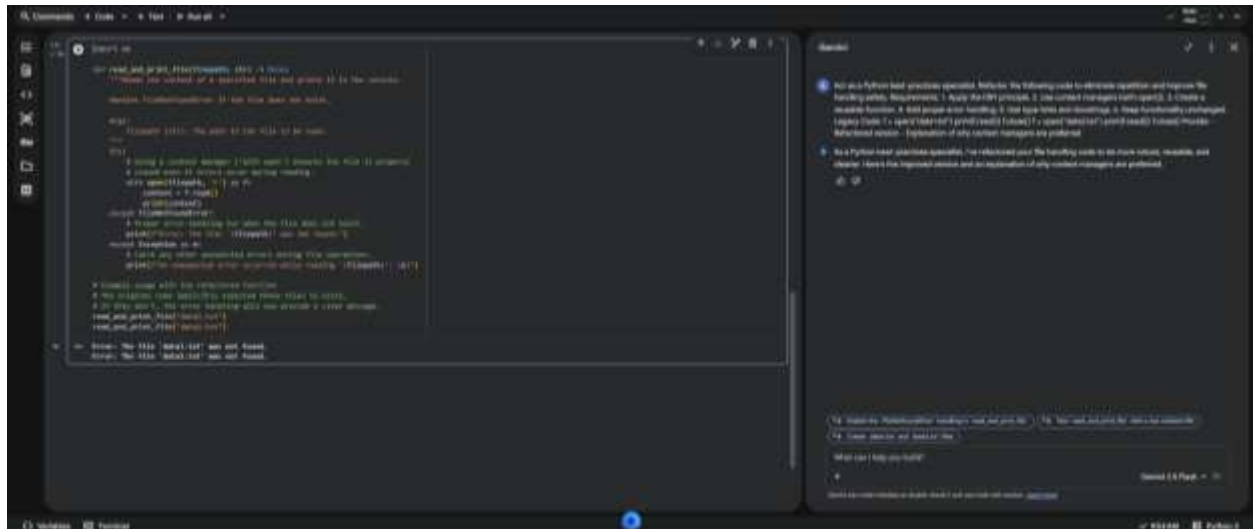
Legacy Code:

```
f = open("data1.txt")
print(f.read())
f.close()

f = open("data2.txt")
print(f.read())
f.close()
```

Provide:

- Refactored version
- Explanation of why context managers are preferred



Task 4 (Optimizing Search Logic)

- Task: Refactor inefficient linear searches using appropriate data structures.
- Focus Areas:
 - o Time complexity
 - o Data structure choice

Legacy Code:

```
users = ["admin", "guest", "editor", "viewer"]

name = input("Enter username: ")

found = False

for u in users:

    if u == name:

        found = True

print("Access Granted" if found else "Access Denied")
```

Expected Outcome:

- o Use of sets or dictionaries with complexity justification.

Prompt : Act as a Python performance optimization expert.

Refactor the following inefficient search implementation.

Requirements:

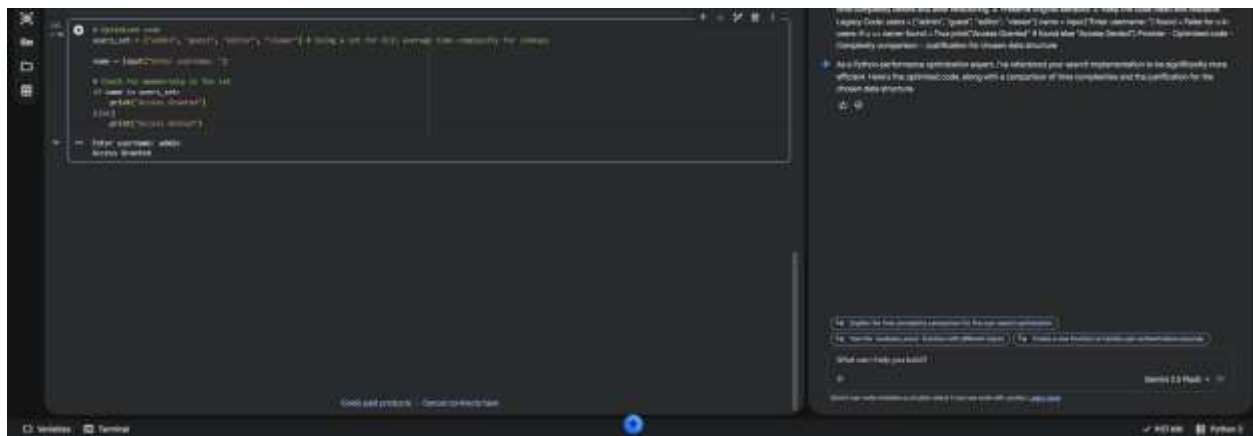
1. Replace linear search with a more efficient data structure.
2. Improve time complexity.
3. Explain time complexity before and after refactoring.
4. Preserve original behavior.
5. Keep the code clean and readable.

Legacy Code:

```
users = ["admin", "guest", "editor", "viewer"]  
  
name = input("Enter username: ")  
  
found = False  
  
for u in users:  
    if u == name:  
        found = True  
  
print("Access Granted" if found else "Access Denied")
```

Provide:

- Optimized code
- Complexity comparison
- Justification for chosen data structure



Task 5 (Refactoring Procedural Code into OOP Design)

- Task: Use AI to refactor procedural code into a class-based design.

- Focus Areas:

- o Object-Oriented principles

- o Encapsulation

Legacy Code:

```
salary = 50000
```

```
tax = salary * 0.2
```

```
net = salary - tax
```

```
print(net)
```

Expected Outcome:

- o A class like EmployeeSalaryCalculator with methods and attributes.

Prompt : Act as an object-oriented design expert.

Refactor the following procedural code into a clean class-based OOP design.

Requirements:

1. Create a class named EmployeeSalaryCalculator.

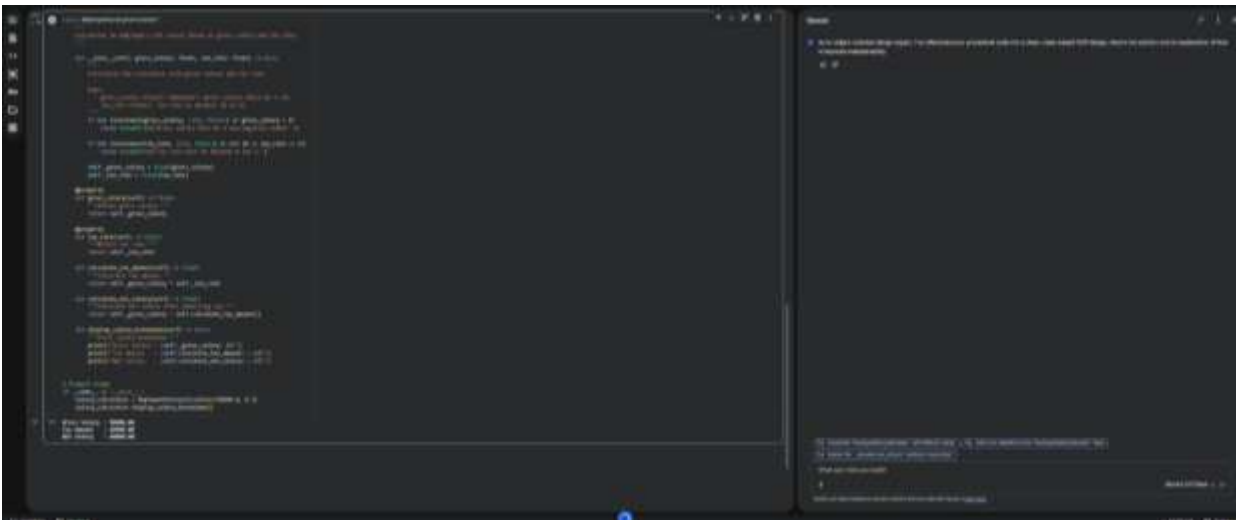
2. Use encapsulation.
3. Include methods for tax calculation and net salary calculation.
4. Use constructor initialization.
5. Add docstrings and type hints.
6. Keep output identical.
7. Explain how OOP improves maintainability.

Legacy Code:

```
salary = 50000  
tax = salary * 0.2  
net = salary - tax  
print(net)
```

Provide:

- Class-based solution
- Explanation of design improvements



Task 6 (Refactoring for Performance Optimization)

- Task: Use AI to refactor a performance-heavy loop handling large data.

- Focus Areas:

- o Algorithmic optimization
- o Use of built-in functions

Legacy Code:

```
total = 0

for i in range(1, 1000000):

    if i % 2 == 0:

        total += i

print(total)
```

Expected Outcome:

- o Optimized logic using mathematical formulas or comprehensions, with time comparison.

Prompt : Act as a Python algorithm optimization expert.

Refactor the following performance-heavy loop.

Requirements:

1. Optimize using mathematical formulas OR built-in Python functions.
2. Improve time complexity.
3. Compare old vs new time complexity.
4. Ensure output remains identical.
5. Explain why the new approach is faster.

Legacy Code:

```
total = 0

for i in range(1, 1000000):

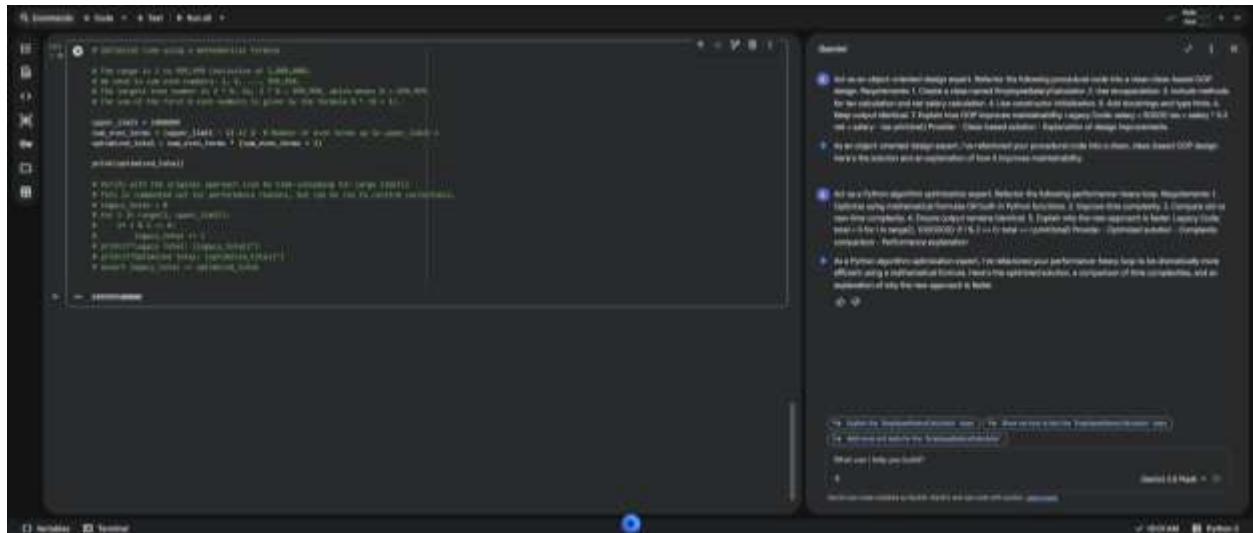
    if i % 2 == 0:

        total += i

print(total)
```

Provide:

- Optimized solution
- Complexity comparison
- Performance explanation



Task 7 (Removing Hidden Side Effects)

- Task: Refactor code that modifies shared mutable state.

- Focus Areas:

o Functional-style refactoring

o Predictability

Legacy Code:

```
data = []
```

```
def add_item(x):
```

```
    data.append(x)
```

```
    add_item(10)
```

```
    add_item(20)
```

```
    print(data)
```

Expected Outcome:

o Refactored function returning values instead of mutating globals.

Prompt :

Act as a functional programming expert.

Refactor the following code to eliminate hidden side effects caused by shared mutable state.

Requirements:

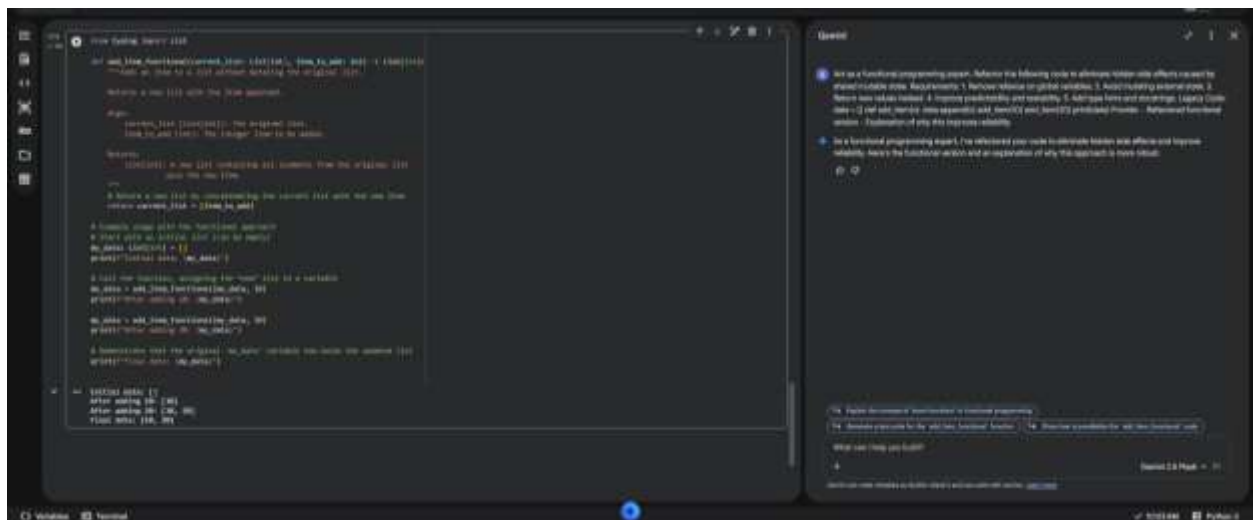
1. Remove reliance on global variables.
2. Avoid mutating external state.
3. Return new values instead.
4. Improve predictability and testability.
5. Add type hints and docstrings.

Legacy Code:

```
data = []  
  
def add_item(x):  
    data.append(x)  
  
add_item(10)  
add_item(20)  
  
print(data)
```

Provide:

- Refactored functional version
- Explanation of why this improves reliability



Task 8 (Refactoring Complex Input Validation Logic)

- Task: Use AI to simplify and modularize complex validation rules.

• Focus Areas:

- o Readability
- o Testability

Legacy Code:

```
password = input("Enter password: ")
```

```
if len(password) >= 8:
```

```
    if any(c.isdigit() for c in password):
```

```
        if any(c.isupper() for c in password):
```

```
            print("Valid Password")
```

```
        else:
```

```
            print("Must contain uppercase")
```

```
        else:
```

```
            print("Must contain digit")
```

```
        else:
```

```
print("Password too short")
```

Expected Outcome:

o Separate validation functions with clear responsibility.

Prompt : Act as a clean architecture and validation expert.

Refactor the following complex password validation logic.

Requirements:

1. Break validation into separate functions.
2. Avoid deeply nested conditionals.
3. Follow Single Responsibility Principle.
4. Improve readability and testability.
5. Maintain identical validation behavior.
6. Add type hints and docstrings.

Legacy Code:

```
password = input("Enter password: ")
if len(password) >= 8:
    if any(c.isdigit() for c in password):
        if any(c.isupper() for c in password):
            print("Valid Password")
        else:
            print("Must contain uppercase")
    else:
        print("Must contain digit")
else:
    print("Password too short")
```

Provide:

- Modular refactored version
- Explanation of improvements

